

Notas de Clase (19 de junio de 2026): Flutter Layouts App

Notas de Clase: Flutter Layouts App

Estructura de un Layout Básico y Modularización

El profesor inició un nuevo proyecto llamado `flutter\_layouts\_app` enfocado en la construcción de interfaces (Layouts).

Puntos clave observados en la pantalla `basic\_design.dart`:

- **Scroll con física personalizada:** Se envolvió el cuerpo de la aplicación en un `SingleChildScrollView` y se le aplicó la propiedad `physics: BouncingScrollPhysics()`. Esto permite que la pantalla se deslice y tenga un efecto visual de rebote al llegar a los bordes, mejorando la experiencia de usuario.
- **Modularización del código (Buenas Prácticas):** Para mantener el código limpio y no saturar el método `build` principal, el diseño se dividió en partes más pequeñas. Dentro de la `Column` principal, en lugar de escribir todo el código de los botones o textos, se hace el llamado a widgets personalizados privados como `\_Title()` y `\_ButtonSection()`, los cuales se construyen más abajo en el mismo archivo.
- **Manejo de Imágenes y Márgenes:** Se utilizó el widget `Image` extrayendo una imagen local (`AssetImage`) con un alto fijo (`height: 250`). Adicionalmente, se implementó un `Container` con márgenes simétricos (`margin: EdgeInsets.symmetric(horizontal: 30, vertical: 20)`) para darle un respiro al bloque de texto inferior y evitar que quede pegado a los bordes de la pantalla.

Documentación Oficial y Widgets de Scroll:

El profesor mostró la documentación oficial de Flutter (docs.flutter.dev) en la sección de Scrolling. Se hizo énfasis en los widgets `ListView` y `GridView`, los cuales son esenciales para implementar "Infinite scrolling" (scroll infinito). La ventaja principal de estos widgets es que construyen los elementos bajo demanda ("on demand") a medida que van apareciendo en la pantalla, lo que mejora drásticamente el rendimiento de la aplicación al manejar listas muy largas. También se mencionaron widgets especializados con comportamientos específicos como el `DraggableScrollableSheet`.

Introducción a PageView y Scroll Vertical:

El profesor creó una nueva pantalla (`scroll_design.dart`) para introducir el widget `PageView`. Este widget es ideal para deslizar pantallas completas una a la vez (estilo TikTok). Aunque por defecto el deslizamiento es horizontal, se destacó el uso de la propiedad `scrollDirection` (específicamente con `Axis.vertical`) para cambiar el comportamiento y permitir que el scroll sea hacia abajo. También se empezó a maquetar la primera vista (`PaginaUno`) usando un `Container` con `width: double.infinity` para asegurar que ocupe todo el ancho de la pantalla de forma adaptativa.

Uso del widget Stack e Imágenes a Pantalla Completa:

El profesor continuó con la `PaginaUno` e introdujo el widget `Stack`. Este widget es fundamental para superponer elementos, es decir, colocar un widget encima de otro como si fueran capas. Dentro de este `Stack`, colocó un contenedor base que incluye un color de fondo azul y una imagen local (`Image.asset('assets/scroll-1.png')`). Para asegurar que el contenedor se expanda verticalmente, le asignó la propiedad `height: double.infinity`, logrando que el diseño cubra la pantalla completa del emulador. Además, el `PageView` principal ya tiene habilitado el scroll vertical y lista las dos pantallas: `PaginaUno()` y `PaginaDos()`.

Agregando Contenido al Stack y Uso del DevTools:

El profesor añadió una `Column` dentro del `Stack`, lo que permite que el texto quede flotando perfectamente sobre la imagen de fondo. Para no repetir código, se declaró una variable `'estilo'` con la configuración de la fuente (tamaño 50 y negrita). Dentro de la columna se incluyeron los textos `'25°C'` y `'Viernes'`, seguidos de un icono grande indicativo de scroll (`keyboard_arrow_down_rounded`). También se mostró en acción el Flutter Inspector (DevTools), visible en el panel izquierdo y a través de los recuadros azules sobre el emulador. Esta herramienta es vital para depurar la interfaz, ya que permite visualizar los límites exactos de cada widget y comprender cómo se posicionan los elementos (como el uso de `mainAxisAlignment` para centrar).

Fondos con Degradados (LinearGradient) y Física de Scroll:

El profesor mejoró el aspecto visual de la pantalla principal envolviendo el PageView dentro de un Container. A este Container se le aplicó la propiedad "decoration" usando un BoxDecoration para implementar un fondo con degradado (LinearGradient). Se definió que el gradiente comience en la parte superior central (begin: Alignment.topCenter) y termine en la inferior central (end: Alignment.bottomCenter), utilizando un arreglo de dos colores hexadecimales (tonos verde agua y azul). Finalmente, al PageView se le agregó la propiedad "physics: BouncingScrollPhysics()" para mantener el efecto de rebote elástico al hacer scroll vertical entre las pantallas.

Creación del HomeScreen y Gradientes con Stops:

El profesor creó una nueva pantalla llamada home_screen.dart y armó una estructura utilizando Stacks anidados (un Stack dentro de otro Stack), lo que indica que se construirán múltiples capas de widgets superpuestos. Para el fondo de esta pantalla, volvió a implementar un LinearGradient dentro de un BoxDecoration, pero introdujo una propiedad nueva clave: 'stops: [0.2, 0.8]'. Esta propiedad recibe un arreglo de números decimales (del 0.0 al 1.0) que indica exactamente en qué porcentaje de la pantalla debe ocurrir la transición de cada color. Esto permite un control preciso del difuminado, creando divisiones de color más marcadas o bloques de color específicos, tal como se observa en el fondo oscuro del emulador.

Posicionamiento Absoluto y Transformaciones (Positioned y Transform):

El profesor refactorizó el código extrayendo el fondo a un widget personalizado llamado 'Gradiente()' para mantener limpio el árbol principal. Luego, introdujo dos herramientas de diseño avanzado dentro del Stack:

1. Widget Positioned: Permite colocar un elemento en coordenadas exactas. Al usar valores negativos (top: -100, left: -40), logró empujar la caja fuera de los límites normales de la pantalla.
2. Widget Transform.rotate: Lo usó para inclinar el elemento (angle: 70).

El elemento hijo es un Container de 350x350 con bordes súper redondeados (BorderRadius.circular(80)) y un gradiente rosado. La combinación de Positioned, Transform y

`borderRadius` genera una figura abstracta y moderna en la esquina superior izquierda, ideal para fondos de aplicaciones.

Refactorización del Fondo y Ajuste de Paleta de Colores:

Se realizó un pequeño ajuste visual y de estructura. El profesor cambió el nombre del widget personalizado a `'_Background()'` (usando el guion bajo al inicio para hacerlo un widget privado, lo cual es una excelente práctica en Dart para encapsular código). Además, afinó la paleta de colores del gradiente principal utilizando códigos hexadecimales exactos (`0xff2E305F` y `0xff202333`), logrando un fondo oscuro con tonos púrpura y azulados que contrasta de manera mucho más elegante y premium con la figura abstracta superior, la cual también adoptó un tono más hacia el violeta/magenta.

Superposición de Contenido y Uso de `SafeArea`:

El profesor comenzó a agregar el contenido de texto sobre el fondo abstracto aprovechando la estructura del `Stack`. El segundo elemento del `Stack` es un `SingleChildScrollView` (para asegurar que el contenido se pueda deslizar si crece mucho). Dentro de este, introdujo un widget vital: `SafeArea`. Este widget protege el contenido para que no se superponga con elementos del sistema operativo (como el "notch" de la cámara o la barra de estado superior). Se le aplicó la propiedad `"bottom: false"` para permitir que el diseño sí se extienda hasta el ras de la pantalla en la parte inferior. Finalmente, usó un `Container` con márgenes y una `Column` alineada a la izquierda (`crossAxisAlignment: CrossAxisAlignment.start`) para colocar el título "Pantalla de GridView" y su subtítulo, los cuales flotan perfectamente sobre la figura rosada del fondo.

Efecto Cristal (Glassmorphism) con `BackdropFilter`:

El profesor introdujo uno de los efectos más modernos en diseño de interfaces: el efecto cristal o 'Glassmorphism'. Para maquetar los elementos en forma de cuadrícula, comenzó utilizando el widget `Table` y `TableRow`. Para lograr que la tarjeta se vea como un vidrio esmerilado, combinó tres herramientas clave:

1. `ClipRRect`: Se encarga de recortar y redondear los bordes del efecto (usando

`BorderRadius.circular(20))`.

2. `BackdropFilter`: Aplica el filtro a todo lo que está dibujado detrás de él. En este caso, usó `ImageFilter.blur(sigmaX: 5, sigmaY: 5)` para lograr el desenfoque perfecto.

3. Un `Container` interno con un color negro semi-transparente (`Colors.black12`) para darle ese tinte de "vidrio oscuro".

El resultado en el emulador es una tarjeta translúcida bellísima que deja ver los colores del fondo distorsionados, muy al estilo de iOS.

Contenido de la Tarjeta (`CircleAvatar` y `Column`):

Dentro de la tarjeta con efecto cristal, el profesor agregó una `Column` centrada usando `'mainAxisAlignment: MainAxisAlignment.center'`. En el centro colocó lo que visualmente actúa como un botón o icono principal: un widget `CircleAvatar` con fondo morado (`Colors.purple`) que contiene un icono de estrella blanca (`Icons.star_rounded`). También se observa que ajustó el color del fondo del contenedor a `Colors.black26` para darle un poco más de oscuridad al vidrio esmerilado y hacer que el icono resalte más.

Reusabilidad de Componentes (`_Card`) y Maquetación con `Table`:

El profesor aplicó una excelente práctica de refactorización al convertir todo el código complejo de la tarjeta de cristal en un widget personalizado y privado llamado `'_Card'`. Para hacer este componente dinámico, lo configuró para recibir argumentos: `color`, `title` (título) e `icon` (ícono). En el árbol de widgets principal, utilizó el widget `Table` estructurado con varios `TableRow` para organizar estas tarjetas en una cuadrícula perfecta (dos por fila). Esto reduce drásticamente las líneas de código, manteniendo el archivo limpio, y permite agregar nuevas tarjetas a la pantalla simplemente llamando a `'_Card(color: ..., title: ..., icon: ...)'` con los datos deseados.